

Angular HandsOn – 6 : Services & Dependency Injection

Aim:

To understand and implement Angular Services and Dependency Injection by creating reusable services, sharing data between components, and demonstrating singleton and component-level service instances in the Student Course Portal application.

Objective:

- Create reusable Angular services.
- Implement Dependency Injection using Angular.
- Understand singleton services using providedIn: 'root'.
- Share data between multiple components.
- Demonstrate service-to-service injection.
- Create component-level providers.

Software Requirements:

- Visual Studio Code
- Node.js
- Angular CLI
- Angular 20
- Google Chrome

Components Created:

- Home Component
- Course List Component
- Course Card Component
- Course Summary Widget Component
- Student Profile Component
- Notification Component

Services Created:

- CourseService
- EnrollmentService
- NotificationService

Model Created:

Course Model (course.model.ts)

id

name

code

credits

gradeStatus

Features Implemented:

Task 1:

- Created CourseService.
- Added five course objects.
- Implemented:
 - getCourses()
 - getCourseById()
 - addCourse()
- Injected CourseService into HomeComponent.
- Injected CourseService into CourseListComponent.
- Displayed total number of available courses.
- Displayed course summary using CourseSummaryWidget.

Task 2:

- Created EnrollmentService.
 - Implemented:
 - enroll()
 - unenroll()
 - isEnrolled()
 - getEnrolledCourses()
 - Injected CourseService inside EnrollmentService.
 - Used EnrollmentService in CourseCardComponent.
 - Implemented Enroll/Unenroll functionality.
 - Displayed enrolled courses in StudentProfileComponent.
 - Created NotificationService using component-level providers.
-

Dependency Injection Used:

Singleton Service:

CourseService and EnrollmentService were provided using
providedIn: 'root'

which creates a single shared instance throughout the application.

Component-Level Provider:

NotificationService was provided inside NotificationComponent using
providers: [NotificationService]

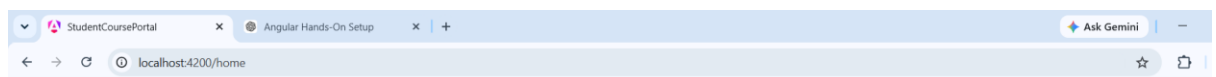
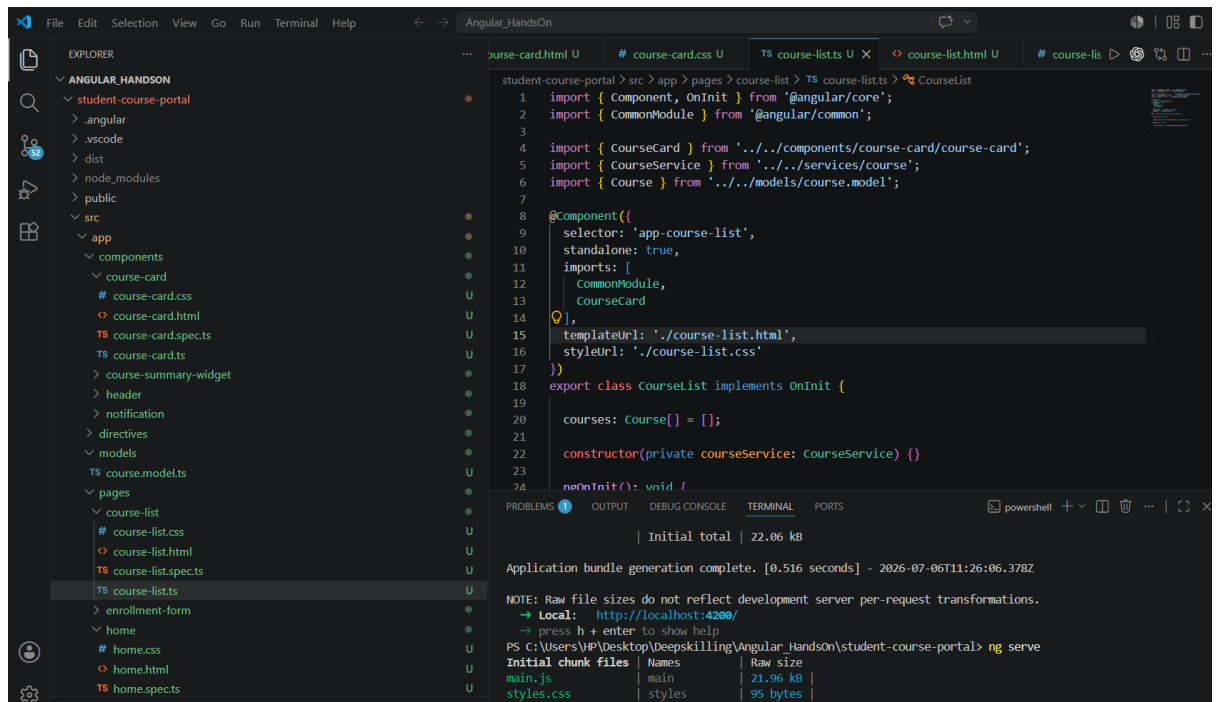
which creates a separate service instance for that component.

Project Structure:

```
src
├── components
│   ├── course-card
│   ├── course-summary-widget
│   └── notification
├── pages
│   ├── home
│   ├── course-list
│   └── student-profile
├── models
│   └── course.model.ts
└── services
    ├── course.service.ts
    ├── enrollment.service.ts
    └── notification.service.ts
```

Output:

- Home page displays total number of courses.
- Course list is loaded using CourseService.
- Enroll button changes to Unenroll after enrollment.
- Student Profile displays enrolled courses.
- Notification component displays its own service instance.
- Data is shared across components using singleton services.



Student Course Portal

[Home](#)[Courses](#)[Profile](#)[Notification](#)

Student Course Portal

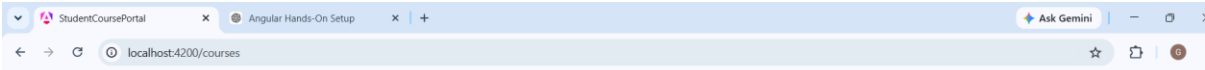
Total Courses Available

6

Course Summary

Total Courses : 6

[Add Course](#)



Angular

Course Code: ANG101

Credits: 4

Status: passed

Unenroll

React

Course Code: REA102

Credits: 3

Status: pending

Unenroll

Node

Course Code: NOD103

Credits: 4

Status: failed

Enroll

Java

Course Code: JAVA104

Credits: 2

Status: passed

Enroll

Python

Course Code: PY105

Credits: 3

Status: pending

Enroll

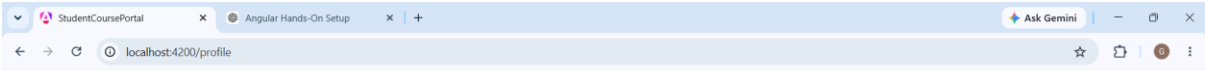
New Course 6

Course Code: NEW6

Credits: 3

Status: pending

Enroll



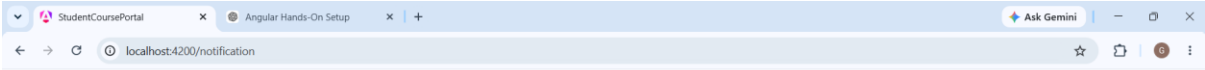
Student Course Portal

[Home](#)[Courses](#)[Profile](#)[Notification](#)

Student Profile

Enrolled Courses

Angular (ANG101)
React (REA102)
Node (NOD103)



Student Course Portal

[Home](#)[Courses](#)[Profile](#)[Notification](#)

Notification

This message is stored in this component instance.

Update Message

Clear

Result

Successfully implemented Angular Services and Dependency Injection using singleton services, service-to-service injection, shared data storage, and component-level providers in the Student Course Portal application.